MTRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Faculty Name	Sanjeev Khatri		
Subject Name with Code	4AD4-06 Introduction to Java Programming		
Semester	4th	**Session**	2022-23 (Even)
Branch	AIDS	**Batch**	A
Unit No.	3	**Submission Date**	17-Mar-23

Lecture No.	*Topic Covered*	*Page No.*	*Total Pages*
1	Inheritance basics, using super and final	1 - 5	5
2	Method overriding, dynamic method dispatch, Abstract Class	6 - 9	4
3	Interface: variables and extending Interfaces	10 - 14	5
4	Package: Creating and importing packages	15 - 18	4
5	Package access protection	19 - 21	3
6	Exception Handling: Exception handling fundamentals, Exception types	22 -23	2
7	Uncaught Exceptions using try and catch	24 - 24	1
8	Multiple catch clauses	25 - 25	1
9	Nested try statements, throw	26 - 30	5
10	Java's built-in exceptions	31 -32	2

Faculty HOD Dean Academics

UNIT-3 (INHERITANCE)

LECTURE - 1

Inheritance Basics, Using Super and Final

Inheritance Basics

Inheritance is one of the three foundation principles of objectoriented programming because it allows the creation of hierarchical classifications. Using inheritance, we can create a general class that defines traits common to a set of related items. This class can then be inherited by other, more specific classes, each adding those things that are unique to it. In Java, a class that is inherited is called a superclass, parent class or base class. The class that does the inheriting is called a subclass, child class or derived class.

We use **extends** keyword to do inheritance in Java. Thus, the subclass or child class adds to (extends) the superclass or parent class.

Types of Inheritance

Types of Inheretence:

- Single Inheretence
- Hierarichal Inherintence
- Multiple Inherintence
- Multilevel Inherintence
- Hybrid Inherintence

Single Inherintence - Derivation a subclass from only one super class is called Single Inherintence.

Hierarchical Inherintence - Derivation of several classes from a single super class is called Hierarchical Inherintence.

Multilevel Inheritance - Derivation of a classes from another derived classes called Multilevel Inheritance.

Multiple Inheritance - Derivation of one class from two or more super classes is called Multiple Inheritance.

But java does not support Multiple Inheritance directly. It can be implemented by using interface concept.

Hybrid Inheritance - Derivation of a class involving more than one from on Inheritance is called Hydrid Inheritance.

Example-

```
class InherTest
{
    public static void main (String args[])
    {
        BedRoom room1  = new BedRoom(14,12,10);
        int area1 = room1.area();
        int volume1 = room1.volume();
        System.out.println("Area = " + area1);
        System.out.println("Volume1 = " + volume1);
    }
}

class Room
{
    int length;
    int breadth;

    Room(int x, int y)
    {
        length = x;
        breadth = y;
    }

    int area()
    {
        return (length * breadth);
    }
}

class BedRoom extends Room
{
    int height;

    BedRoom(int x, int y, int z)
    {
        super(x,y);
        height = z;
    }

    int volume()
    {
        return (length * breadth * height);
    }

}
```

Using Super to call a constructor of superclass

A subclass can call a constructor defined by its superclass by use of the following form of **super**:

super(parameterlist);

parameterlist specifies any parameters needed by the constructor in the superclass. super( ) must always be the first statement executed inside a subclass constructor.

Using Super to access superclass members

There is a second form of super that acts somewhat like this, except that it always refers to the superclass of the subclass in which it is used. This usage has the following general form:

super.member

Here, member can be either a method or an instance variable.

Using the @Override Tag

When we override a parent class method in a child class, you can insert an override annotation just prior to the method. The override annotation lets the compiler know that our intention is to override a method in the parent class rather than create a method with a new signature.

Using Final Keyword

As powerful and useful as method overriding and inheritance are, sometimes we need to prevent them. In Java it is easy to prevent a method from being overridden or a class from being inherited by using the keyword **final**. To prevent a method from being overridden, specify **final** as a modifier at the start of its declaration. Methods declared as final cannot be overridden. We can prevent a class from being inherited by preceding its declaration with **final**. Declaring a class as final implicitly declares all of its methods as **final**, too.

We can use final key word in three ways:
Used to create equivalent of a named constant
Final datatype identifier = ... ; →Used to prevent inheritance
Final class ……..
Used to avoid overloading
Final return type ……

Using final to Prevent Overriding -

To disallow a method from being overridden, specify final as a modifier at the start of its declaration. Methods declared as final cannot be overridden.

Example-

```
public class finaltoprevover
{
    public static void main(String args[])
    {

    }
}

class parentclass
{
    final void show()
    {
        System.out.println("This is a final method.");
    }
}

class childclass extends parentclass
{
    void show() //Cannot override the final method from parentclass
    {
        System.out.println("Not allowed");
    }
}
```

Using final to Prevent Inheritance -

Sometimes we will want to prevent a class from being inherited. To do this, we declare the class as final. Declaring a class as final implicitly declares all of its methods as final, too.

Example-

```
public class finaltoprevover
{
    public static void main(String args[])
    {

    }
}

final class parentclass
{
    void show()
    {
        System.out.println("This is a final method.");
```

```
    }
}

class childclass extends parentclass
{
    void show() //Cannot override the final method from parentclass
    {
        System.out.println("Not allowed");
    }
}
```

Output-

```
error: cannot inherit from final parentclass
class childclass extends parentclass
                ^
1 error
```

MITRC

LECTURE - 2

Method Overriding, Dynamic Method Dispatch, Abstract Class

Method Overriding

- In a class hierarchy, when a method in a sub class has the same name and type signature as a method in its super class, then the method in the sub class is said to be override the method in the super class.
- When an overridden method is called from within a sub class, it will always refers to the version of that method defined by the sub class.
- The version of the method defined in the super class is hidden.

Example-

```
public class methodoverride1 {
    public static void main(String args[])
    {
        childclass cc = new childclass(5,6,7);
        cc.show();
    }
}

class parentclass{
    int i,j;
    parentclass(int a, int b)
    {
        i=a;
        j=b;
    }

    void show()
    {
        System.out.println("Value of i is : " + i + " and j is : " + j);
    }
}

class childclass extends parentclass
{
    int k;
    childclass(int a, int b, int c)
    {
        super(a, b);
        k=c;
    }
```

```
    void show()

    {
        super.show();
        System.out.println("Value of K is : " + k);
    }
}
```

Output-
Value of i is : 5 and j is : 6
Value of K is : 7

Method overriding occurs only when the signatures of the two methods are identical. If they are not, then the two methods are simply overloaded.

Dynamic Method Dispatch

- ❖ It is a mechanism by which a call to an overridden method is resolved at run time rather then compile time.
- ❖ It is important because this is how java implements runtime polymorphism.

Example-

```
public class dynamicmthdispatch {
    public static void main(String args[])
    {
        Sup supob = new Sup();
        Sub1 sub1ob = new Sub1();
        Sub2 sub2ob = new Sub2();
        Sup supRef;

        supRef = supob;
        supRef.who();
        supRef = sub1ob;
        supRef.who();
        supRef = sub2ob;
        supRef.who();
    }
}


class Sup
{
    void who()
```

```
    {
        System.out.println("who() in Sup");
    }
}

class Sub1 extends Sup
{
    void who()
    {
        System.out.println("who() in Sub1");
    }
}

class Sub2 extends Sup
{
    void who()
    {
        System.out.println("who() is in Sub2");
    }
}
```

Abstract Class

An abstract method is a method that is declared with only its signatures with out implementations. An abstract class is class that has at least one abstract method. Since an abstract class does not define a complete implementation, there can be no objects of an abstract class. Thus, attempting to create an object of an abstract class by using new will result in a compiletime error.

The syntax is:

Abstract class class-name
{
    Variables
    Abstract methods;
    Concrete methods;
    .
    .
    .
}

Important Points regarding Abstract Classes-

- ❖ An abstract class is one from which you cannot inherit, but from which you can create concrete objects.
- ❖ Abstract classes usually have one or more empty abstract methods.

❖ An abstract method has no body, curly braces, or statements.

Example-

```
abstract class baseclass
{
    abstract void showme();
    // concrete methods are still allowed in abstract classes
    void showmetoo()
    {
        System.out.println("showmetoo is a concrete method.");
    }
}

class childclass extends baseclass
{
    void showme()
    {
        System.out.println("Implementation of showme in childclass.");
    }
}

class abstractclassdemo
{
    public static void main(String args[])
    {
        childclass child = new childclass();
        child.showme();
        child.showmetoo();
    }
}
```

Output-

Implementation of showme in childclass.
showmetoo is a concrete method.

LECTURE - 3

Interface: Variables and Extending Interfaces

Interface

An interface defines a set of methods that will be implemented by a class. Thus, an interface gives us a way to specify what a class will do, but not how it will do it. In Java, we can fully separate a class's interface from its implementation by using the keyword interface.

An interface is syntactically similar to an abstract class, in that we can specify one or more methods that have no body. Those methods must be implemented by a class in order for their actions to be defined. Once an interface is defined, any number of classes can implement it. Also, one class can implement any number of interfaces.

How to Implement

To implement an interface, a class must provide bodies (implementations) for the methods described by the interface. Each class is free to determine the details of its own implementation.

Syntex-

```
access_specifier interface name
{
        rettype methodname1(paramlist);
        rettype methodname2(paramlist);
        type var1 = value;
        type var2 = value;
        // ...
        rettype methodnameN(paramlist);
        type varN = value;
}
```

Once an interface has been defined, one or more classes can implement that interface. To implement an interface, include the **implements** clause in a class definition and then create the methods required by the interface. The general form of a class that includes the implements clause looks like this:

```
class classname extends superclass implements interface
{
```

MITRC

```
        // classbody
}
```

To implement more than one interface, the interfaces are separated with a comma and the extends clause is optional.

Example-

```
import java.util.*;

interface Area
{
    final static float pi = 3.14F;
    float compute (float x, float y);
}

class Rectangle implements Area
{
    public float compute (float x, float y)
    {
        return (x*y);
    }
}

class Circle implements Area
{
    public float compute(float x, float y)
    {
        return (pi * x * x);
    }
}

class InterfaceTest
{
    public static void main(String args[])
    {
        Rectangle rect = new Rectangle();
        Circle cir = new Circle();
        Area area;
        area = rect;
        System.out.println("Area of Rectangle  = " + area.compute(10,20));
        area = cir;
        System.out.println("Area of Circle = " + area.compute (10,0));
    }
}
```

Important Points about Interfaces-

- ❖ Interface is a collection of method declarations and constants that one or more classes of Non objects will use.
- ❖ We can implement multiple inheritance using interface.
- ❖ Because interface consists only signatures followed by semicolon and parameter list, they are implicitly abstract.
- ❖ Variables can be declared and initialized inside interface they are implicitly final and static.
- ❖ An interface method can't be final or static.
- ❖ An interface can be extended from another interface.

Variables in Interfaces

Variables can be declared in an interface, but they are implicitly public, static, and final. We might think that there would be very limited use for such variables, but it's not true. Large programs typically make use of several constant values that describe such things as array size, various limits, special values, and the like.

Example-

```
public class sharedconst implements sharedconstants {
    public static void main(String args[])
    {
        int nos[] = new int[Max];
        for(int i=Min; i<11;i++)
        {
            if(i >= Max)
            {
                System.out.println(Error);
            }
            else
            {
                nos[i] = i;
                System.out.println(nos[i] + " ");
            }

        }
    }
}

interface sharedconstants
{
    int Min = 0;
    int Max = 10;
    String Error = "Beyond the Limits";
}
```

Output-

0
1
2
3
4
5
6
7
8
9
Beyond the Limits

Extending Interface-

One interface can inherit another by use of the keyword extends. The syntax is the same as for inheriting classes. When a class implements an interface that inherits another interface, it must provide implementations for all methods defined within the interface inheritance chain.

Example-

```
public class extendinginterface
{
    public static void main(String arg[])
    {
        otherclass oc = new otherclass();
        oc.method1();
        oc.method2();
        oc.method3();
    }
}

interface first
{
    void method1();
    void method2();
}

// B now includes method1() and method2() -- it will add method3().
interface second extends first
{
    void method3();
}

// This class will implement all the methods of first and second
class otherclass implements second
```

```
{
    public void method1()
    {
        System.out.println("Implementing method1()");
    }
    public void method2()
    {
        System.out.println("Implementing method2()");
    }
    public void method3()
    {
        System.out.println("Implementing method3()");
    }
}
```

Output-

Implementing method1()
Implementing method2()
Implementing method3()

MITRC

LECTURE - 4

Package: Creating and Importing Packages

Packages

In programming, it is often helpful to group related pieces of a program together. In Java, this can be accomplished by using a package. A package serves two purposes. First, it provides a mechanism by which related pieces of a program can be organized as a unit. Second, a package participates in Java's access control mechanism. Classes defined within a package can be made private to that package and not accessible by code outside the package. Thus, the package provides a means by which classes can be encapsulated. Packages are collection of related classes and works like a container.

In a package, you can define code that is accessible by other code within the same package but not by code outside the package. This enables you to create selfcontained groups of related classes that keep their operation private.

Defining a Package

To create a package, we neet to put a **package** command at the top of a Java source file. The classes declared within that file will then belong to the specified package. Since a package defines a namespace, the names of the classes that we put into the file become part of that package's namespace.

Syntex-
package pkgname;

More than one file can include the same package statement. The package statement simply specifies to which package the classes defined in a file belong. We can create a hierarchy of packages. To do so, we neet to simply separate each package name from the one above it by use of a period. Like-

package pack1.pack2.pack3...packN;

Importing a Package

Java includes the import statement to bring certain classes, or entire packages, into visibility. Once imported, a class can be referred to directly, using only its name. The import statement is a convenience to the programmer and is not technically needed to write a complete Java program. If we are going to refer to a few dozen classes in our application, however, the import statement will save a lot of typing.

Syntex-
import pkg1[.pkg2].(classname|*);

MITRC

Here, pkg1 is the name of a top-level package, and pkg2 is the name of a subordinate package inside the outer package separated by a dot (.). There is no practical limit on the depth of a package hierarchy, except that imposed by the file system. Finally, we specify either an explicit classname or a star (*), which indicates that the Java compiler should import the entire package.

Example-

This is the main program which is importing bookdtls class from BookDetails folder-

```
import BookDetails.bookdtls;

public class packagdemo
{
    public static void main(String args[])
    {
        bookdtls b1=new bookdtls("Subject-1", "Author-1", 2023, "Publisher-1",
1000);
        b1.show();
        bookdtls b2=new bookdtls("Subject-2", "Author-2", 2023, "Publisher-2",
1500);
        b2.show();
    }
}
```

The following is the bookdtls class which is available in BookDetails folder. BookDetails folder working as a package-

```
package BookDetails;
public class bookdtls
{
    public String title;
    public String author;
    public int year;
    public String publisher;
    public int price;
    public bookdtls(String title, String author, int year, String publisher, int
price)
    {
        this.title=title;
        this.author=author;
        this.year=year;
        this.publisher=publisher;
        this.price=price;
    }
```

```
    public void getTitle()
    {
        System.out.println("Title of Book : "+title);
    }

    public void getAuthor()
    {
        System.out.println("Author of Book : "+author);
    }

    public void getYear()
    {
        System.out.println("Year: "+year);
    }

    public void getPublisher()
    {
        System.out.println("Publisher of Book : "+publisher);
    }
    public void getPrice()
    {
        System.out.println("Price of Book : "+price);
    }

    public void setTitle(String title)
    {
        this.title=title;
    }

    public void setAuthor(String author)
    {
        this.author=author;
    }
    public void setYear(int year)
    {
        this.year=year;
    }

    public void setPublisher(String publisher)
    {
        this.publisher=publisher;
    }

    public void setCost(int price)
    {
        this.price=price;
    }

    public void show()
```

```
    {
        getTitle();
        getAuthor();
        getYear();
        getPublisher();
        getPrice();
    }
}
```

Output-

Title of Book : Subject-1
Author of Book : Author-1
Year: 2023
Publisher of Book : Publisher-1
Price of Book : 1000
Title of Book : Subject-2
Author of Book : Author-2
Year: 2023
Publisher of Book : Publisher-2
Price of Book : 1500

MITRC

LECTURE - 5

Package Access Protection

Access Protection

Packages also participate in Java's access control mechanism. The visibility of an element is affected by its access specification—private, public, protected, or default—and the package in which it resides. Thus, as it relates to classes and packages, the visibility of an element is determined by its visibility within a class and its visibility within a package. This multilayered approach to access control supports a rich assortment of access privileges.

Class Member Access

	Private Member	Default Member	Protected Member	Public Member
Visible within same class	Yes	Yes	Yes	Yes
Visible within same package by subclass	No	Yes	Yes	Yes
Visible within same package by non-subclass	No	Yes	Yes	Yes
Visible within different package by subclass	No	No	Yes	Yes
Visible within different package by non-subclass	No	No	No	Yes

Example-

The file calling Book class from bookpack package which is having private and public members.

```
class UseBook {
    public static void main(String args[])
    {
        bookpack.Book books[] = new bookpack.Book[5];
        books[0] = new bookpack.Book("Java : A Beginner's Guide", "Schildt",
2019);
        books[1] = new bookpack.Book("Java : The Complete Reference", "Schildt",
2019);
        books[2] = new bookpack.Book("Introducing JavaFX 8 Programming",
"Schildt", 2015);
        books[3] = new bookpack.Book("Red Strom Rising", "Clancy", 1986);
        books[4] = new bookpack.Book("On the Road", "Kerouac", 1955);
```

```
        for(int i=0; i<books.length; i++)
        {
            books[i].show();
        }
    }
}
```

The file Book class in bookpack package is having private and public members.

```
package bookpack;
public class Book {
    private String title;
    private String author;
    private int pubDate;

    //Now Public
    public Book(String t, String a, int d)
    {
        title = t;
        author = a;
        pubDate = d;
    }

    //Now Public
    public void show()
    {
        System.out.println(title);
        System.out.println(author);
        System.out.println(pubDate);
        System.out.println();
    }
}
```

Output-

Java : A Beginner's Guide
Schildt
2019

Java : The Complete Reference
Schildt
2019

Introducing JavaFX 8 Programming
Schildt
2015

Red Strom Rising
Clancy
1986

On the Road
Kerouac
1955

Protected Members

The protected modifier creates a member that is accessible within its package and to subclasses in other packages. Thus, a protected member is available for all subclasses to use but is still protected from arbitrary access by code outside its package.

MITRC

LECTURE - 6

Exception Handling: Exception Handling Fundamentals, Exception Types

Exceptions

An exception is an unexpected or error condition. The programs we write can generate many types of potential exceptions:

- A program might issue a command to read a file from a disk, but the file does not exist there.
- A program might attempt to write data to a disk, but the disk is full or unformatted.
- A program might ask for user input, but the user enters an invalid data type.
- A program might attempt to divide a value by 0.
- A program might try to access an array with a subscript that is too large or too small.

These errors are called exceptions.

Exception Handling Fundamentals

Exception handling is the name for the object-oriented techniques that manage or resolve such errors. Unplanned exceptions that occur during a program's execution are also called **runtime exceptions**, in contrast with **syntax errors** that are discovered during program compilation. Object-oriented exception-handling techniques provide more elegant and safer solutions for handling errors.

Java exception handling is managed via five keywords: **try**, **catch**, **throw**, **throws**, and **finally**. They form an interrelated subsystem in which the use of one implies the use of another.

Program statements that we want to monitor for exceptions are contained within a **try** block. If an exception occurs within the **try** block, it is thrown. Our code can **catch** this exception using **catch** and handle it in some rational manner. System generated exceptions are automatically thrown by the Java runtime system. To manually throw an exception, we use the keyword **throw**.

Using Try and Catch

At the core of exception handling are **try** and **catch**. These keywords work together. We can't have a **catch** without a **try**.

Syntex-

```
try {
        // block of code to monitor for errors
}
catch (ExcepType1 exOb) {
```


MITRC

```
        // handler for ExcepType1
}
catch (ExcepType2 exOb) {
        // handler for ExcepType2
}
```

Example-

```
public class excepdemo1 {
    public static void main(String args[])
    {
        int nums[] = new int[4];
        try
        {
            System.out.println("Before Exception is Generated");
            nums[7] = 10;
            System.out.println("This won't be displayed");
        }
        catch (ArrayIndexOutOfBoundsException exc)
        {
            //catch the exception
            System.out.println("Index out-of-bonds!");
        }
        System.out.println("After catch statement");
    }
}
```

Output-

Before Exception is Generated
Index out-of-bonds!
After catch statement

Advantages of Exception Handling

1. It makes out programs clearer and more flexible.
2. when a method throws an exception, it will always be handled in the same, consistent way.
3. In many cases, we want a method to check for errors, but we do not want to require the method to handle an error if it finds one.

LECTURE - 7

Uncaught Exceptions Using Try and Catch

Uncaught Exceptions

If our program does not catch an exception, then it will be caught by the JVM. Such exceptions are called uncaught exceptions.

Example-

```
public class excepdemo1 {
    public static void main(String args[])
    {
        int nums[] = new int[4];
        try
        {
            System.out.println("Before Exception is Generated");
            nums[7] = 10;
            System.out.println("This won't be displayed");
        }
        catch (ArithmeticException exc)
        {
            //catch the exception
            System.out.println("Index out-of-bonds!");
        }
        System.out.println("After catch statement");
    }
}
```

Output-

Before Exception is Generated
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 7 out of bounds for length 4 at excepdemo1.main(excepdemo1.java:8)

In above program actual error is Index out-of-bounds. But we are trying to find out Arithmetic Exception. That's why compiler has produced an error during run time which is known as **uncaught exception**.

LECTURE - 8

Multiple Catch Clauses

We can associate more than one **catch** statement with a **try**. It is common to do so. However, each **catch** must catch a different type of exception. In general, catch expressions are checked in the order in which they occur in a program. Only a matching statement is executed. All other catch blocks are ignored.

Example-

```
public class excepdemo
{
    public static void main(String args[])
    {
        try
        {
            int arr[]=new int[7];
            arr[7]=30/3;
            System.out.println("Last Statement of try block");
        }
        catch(ArithmeticException e){
            System.out.println("You should not divide a number by zero");
        }
        catch(ArrayIndexOutOfBoundsException e){
            System.out.println("Accessing array elements outside of the limit");
        }
        catch(Exception e){
            System.out.println("Some Other Exception");
        }
        System.out.println("Out of the try-catch block");
    }
}
```

Output-

Accessing array elements outside of the limit
Out of the try-catch block

In above example we have used more than one catch with on try-catch block to handle different types of exceptions.

LECTURE - 9

Nested Try Statements, Throw

Nested Try Blocks

One try block can be nested within another. An exception generated within the inner try block that is not caught by a catch associated with that try is propagated to the outer try block.

Example-

```
public class NestedTryes {
    public static void main(String args[])
    {
        int number[] = {4, 8, 16, 32, 64, 128, 256, 512};
        int denom[] = {2, 0, 4, 4, 0, 8};
        //Here, number is longer than denom
        try
        {
            for(int i=0; i<number.length;i++)
            {
                //nested try
                try
                {
                    System.out.println(number[i] + " / " + denom[i] + " is " +
number[i]/denom[i]);
                }
                catch(ArithmeticException exc)
                {
                    //catch the exception
                    System.out.println("Can't divide by Zero!");
                }
            }
        }
        catch(ArrayIndexOutOfBoundsException exc)
        {
            //catch the exception
            System.out.println("No matching element found.");
            System.out.println("Fatal error - program terminated");
        }
    }
}
```

```
Output-
4 / 2 is 2
Can't divide by Zero!
16 / 4 is 4
```

```
32 / 4 is 8
Can't divide by Zero!
128 / 8 is 16
No matching element found.
Fatal error - program terminated
```

Throw Keyword

So far, we have only been catching exceptions that are thrown by the Java Run - Time systems. However, it is possible for our program to throw an exception explicitly, using the throw statement - Throw throwableInstance

Here, **ThrowableInstance** must be an object of type Throwable or a subclass of Throwable. Simple types, such as int or char, as well as non-Throwable classes, such as String and Object, cannot be used as exceptions. There are two ways we can obtain a Throwable object:

– using a parameter into a catch clause

– creating one with the new operator.

The flow of execution stops immediately after the throw statement, any subsequent statements are not executed. The nearest enclosing try block is inspected to see if it has a catch statement that matches the type of the exception.

Example-

```
public class ThrowDemo {
    static void demomethod()
    {
        try
        {
            throw new NullPointerException("demo");
        }
        catch(NullPointerException e)
        {
            System.out.println("Caught inside demomethod."); throw e; // rethrow
the exception
        }
    }

    public static void main(String args[])
    {
        try
        {
            demomethod();
        }
        catch(NullPointerException e)
        {
            System.out.println("Recaught: " + e);
        }
    }
}
```

Output-
Caught inside demomethod.
Recaught: java.lang.NullPointerException: demo
In this program **new** is used to construct an instance of **NullPointerException**.

Throws Keyword

If a method is capable of causing an exception that it does not handle, it must specify this behavior so that callers of the method can guard themselves against that exception. We do this by including a throws clause in the method's throws clause lists declar the types of exceptions that a method might throw. It is necessary for all exceptions, except those of type Error or RuntimeException, or any of their subclasses. All other exceptions that a method can throw must be declared in the throws clause. If they are not, a compile-time error will result.

Syntex-
type method-name(parameter-list) throws exception-list
{
// body of method
}
Here, exception-list is a comma-separated list of the exceptions that a method can throw.

Example-

```
public class ThrowsDemo {
    static void throwThis() throws IllegalAccessException
    {
        System.out.println("Inside throwThis.");
        throw new IllegalAccessException("demo");
    }
    public static void main(String args[])
    {
        try
        {
            throwThis();
        }
        catch (IllegalAccessException e)
        {
            System.out.println("Caught " + e);
        }
    }
}
```

Output-
Inside throwThis.
Caught java.lang.IllegalAccessException: demo

Using Finally

Sometimes we want to define a block of code that will execute when a try/catch block is left. However, that method may have opened a file or a network connection that needs to be closed. Such types of circumstances are common in programming, and Java provides a convenient way to handle them: **finally**.

To specify a block of code to execute when a try/catch block is exited, include a finally block at the end of a try/catch sequence.

Syntex-

```
try {
        // block of code to monitor for errors
}
catch (ExcepType1 exOb) {
        // handler for ExcepType1
}
catch (ExcepType2 exOb) {
        // handler for ExcepType2
}
//...
finally {
        // finally code
}
```

The finally block will be executed whenever execution leaves a try/catch block, no matter what conditions cause it. That is, whether the try block ends normally, or because of an exception, the last code executed is that defined by finally. The finally block is also executed if any code within the try block or any of its catch statements return from the method.

Example-

```
class useFinally {
    public static void showException(int what)
    {
        int t;
        int nums[] = new int[2];
        System.out.println("Receiving " + what);
        try
        {
            switch(what)
            {
                case 0:
                    t = 10 / what;
                    break;
                case 1:
                    nums[4] = 4;
```

MITRC

```
                    break;
                case 2:
                    return;
            }
        }
        catch (ArithmeticException exc)
        {
            System.out.println("Can't divide by Zero!");
            return;
        }
        catch (ArrayIndexOutOfBoundsException exc)
        {
            System.out.println("No matching element found.");
        }
        finally
        {
            System.out.println("Leaving try.");
        }
    }

}

public class finallyDemo
{
    public static void main(String args[])
    {
        for(int i=0;i<3; i++)
        {
            useFinally.showException(i);
            System.out.println();
        }
    }
}
```

Output-

Receiving 0
Can't divide by Zero!
Leaving try.

Receiving 1
No matching element found.
Leaving try.

Receiving 2
Leaving try

LECTURE - 10

Java's Built-In Exceptions

Inside the standard package java.lang, Java defines several exception classes. The most general of these exceptions are subclasses of the standard type RuntimeException. Since java.lang is implicitly imported into all Java programs, many exceptions derived from RuntimeException are automatically available. Furthermore, they need not be included in any method's throws list. In the language of Java, these are called unchecked exceptions because the compiler does not check to see if a method handles or throws these exceptions.

The Unchecked Exceptions Defined in java.lang are-

Exception	Meaning
ArithmeticException	Arithmetic error, such as integer divide-by-zero.
ArrayIndexOutOfBoundsException	Array index is out-of-bounds.
ArrayStoreException	Assignment to an array element of an incompatible type.
ClassCastException	Invalid cast.
EnumConstantNotPresentException	An attempt is made to use an undefined enumeration value.
IllegalArgumentException	Illegal argument used to invoke a method.
IllegalCallerException	A method cannot be legally executed by the calling code.
IllegalMonitorStateException	Illegal monitor operation, such as waiting on an unlocked thread.
IllegalStateException	Environment or application is in incorrect state.
IllegalThreadStateException	Requested operation not compatible with current thread state.
IndexOutOfBoundsException	Some type of index is out-of-bounds.
LayerInstantiationException	A module layer cannot be created.
NegativeArraySizeException	Array created with a negative size.
NullPointerException	Invalid use of a null reference.
NumberFormatException	Invalid conversion of a string to a numeric format.
SecurityException	Attempt to violate security.
StringIndexOutOfBoundsException	Attempt to index outside the bounds of a string.
TypeNotPresentException	Type not found.
UnsupportedOperationException	An unsupported operation was encountered.

The Checked Exceptions Defined in java.lang are-

Exception	Meaning
ClassNotFoundException	Class not found.
CloneNotSupportedException	Attempt to clone an object that does not implement the **Cloneable** interface.
IllegalAccessException	Access to a class is denied.
InstantiationException	Attempt to create an object of an abstract class or interface.
InterruptedException	One thread has been interrupted by another thread.
NoSuchFieldException	A requested field does not exist.
NoSuchMethodException	A requested method does not exist.
ReflectiveOperationException	Superclass of reflection-related exceptions.

MITRC